Master's degree in Electronic Engineering

Politecnico di Torino

Operating Systems

Linux scheduler: Enhanching and Experimenting

Final Project Report

Group 11

Mattia Cesaraccio 346408

Francesca Di Giuseppe 344875

Gianmarco Fortunelli 344387

Turin, February 2026

Contents

1 Introduction **2**

2 System Architecture **2**

2.1 Environment Setup . . . 2

2.1.1 Software Stack and Dependencies . . . 2

2.1.2 Differential Workspace Configuration . . . 3

2.2 Experimental Methodology . . . 3

2.2.1 Unified Build and Tracing Pipeline . . . 3

2.2.2 Differential Analysis and Custom Injection . . . 4

2.2.3 Statistical Rigor . . . 4

3 Static vruntime boosting **4**

3.1 Modification . . . 4

3.2 Benchmark . . . 5

3.2.1 Process Archetypes . . . 5

3.2.2 Observability and parsering of results . . . 6

4 Short block preempt **6**

4.1 Modification . . . 6

4.1.1 Parameters Tuning and Benchmark . . . 7

5 Adaptive Per-Core Scheduling Policy **7**

5.1 Per-Task Behavioral Characterization . . . 7

5.2 Per-Core Workload Aggregation . . . 8

5.3 Adaptive Parameter Scaling and Enforcement . . . 8

5.4 Benchmark Adaptation for Phase-Controlled Workloads . . . 9

5.5 Experimental Infrastructure and Automated Analysis . . . 10

5.6 Results of adaptive policy . . . 10

6 Conclusions **11**

1 Introduction

Modern operating systems rely on complex scheduling policies to balance CPU time among competing tasks. The Linux kernel implements this via the Completely Fair Scheduler (CFS), a *vruntime*-based algorithm designed to optimize fairness and throughput. While CFS is effective for general workloads, its internal heuristics can be tuned or modified to prioritize specific performance goals, such as reducing wake-up latency for I/O-bound processes.

The objective of this project is to implement and evaluate a custom modification to the Linux 5.4.281 scheduler compiled for the x86_64 architecture, comparing its impact against the unmodified baseline kernel. Given that scheduler changes require full kernel recompilation, we developed a fully automated, consistent, and reproducible pipeline. This workflow handles the retrieval of sources, the assembly of a minimalist BusyBox-based *initramfs*, and the execution of controlled benchmarks within a QEMU virtual machine, the collection of performance traces, and the subsequent statistical analysis.

To ensure experimental rigor and reproducibility, the environment is strictly controlled: the VM is configured with a single CPU core and with frequency scaling and turbo option disabled, to eliminate non-deterministic interference. This setup, combined with the use of *Ftrace*, allows for high-precision tracing of scheduler events. The following sections detail the automation infrastructure, the specific kernel modifications introduced to "boost" task responsiveness, the benchmarks used, and the results obtained.

2 System Architecture

2.1 Environment Setup

The foundation of the project relies on a scripted environment designed to ensure consistency, automation, and reproducibility. The `fetch.sh` script automates the acquisition of the toolchain and source code, eliminating human error and ensuring that every experiment starts from an identical state.

2.1.1 Software Stack and Dependencies

To maintain a stable and observable environment, we selected specific versions of the kernel and userspace utilities. These are summarized in Table 1.

Component	**Version**	**Role in Project**
Linux Kernel	5.4.281 (LTS)	Target for scheduler modification and *vruntime* analysis.
BusyBox	1.36.1	Minimalist userspace to reduce background noise/jitter.
QEMU	system-x86	Hardware-isolated emulation for safe kernel execution.
Build Tools	GCC, GNU Make	Static compilation of benchmarks and kernel image.
Libraries	libelf, libssl, bc	Required dependencies for modern kernel builds.

Table 1: Software stack and development dependencies.

We opted for the Linux Kernel v5.4.281 (LTS) because it represents a stable maturity of the Completely Fair Scheduler (CFS). This allows our research to focus on the classic *vruntime*-based logic. The use of BusyBox further ensures that the CPU is dedicated almost exclusively to our benchmarks, minimizing interference from background system services.

2.1.2 Differential Workspace Configuration

To isolate the effects of scheduler modifications, the script implements a symmetric directory structure. This creates a strict "control group" environment where all variables except the scheduler logic remain constant:

- **Baseline** (`linux-original`): An unmodified source tree used to establish reference performance metrics.
- **Experimental** (`linux-custom`): The target directory where kernel patches from the `src/` folder are injected.

This differential setup ensures that any observed performance deltas are exclusively attributable to our modifications.

2.2 Experimental Methodology

The core of our experiment relies on a unified execution and tracing pipeline designed to isolate the scheduler's behavior as the sole independent variable. While `build_run_original.sh` and `build_run_custom.sh` operate on distinct kernel trees, they share an identical orchestration logic to ensure a *ceteris paribus* comparison.

2.2.1 Unified Build and Tracing Pipeline

The construction of the experimental environment follows a multi-stage process centered on static linking and kernel observability:

- **Image Construction**: Both BusyBox and the benchmark files are compiled statically to eliminate dynamic library dependencies. These are packaged into a compressed *cpio* `initramfs`, which the kernel loads into RAM at boot time.
- **System Initialization** (`init`): A minimalist `init` script is generated and placed at the root of the filesystem to manage the guest's lifecycle:
  - **Environment Setup**: It mounts essential virtual filesystems, including `/proc`, `/sys`, `/dev`, and `debugfs` (the interface for Ftrace).
  - **Orchestration**: It triggers the execution of the `run_bench.sh` script.
  - **Automated Power-off**: After the benchmarks finish, it performs a `sync` and a `poweroff -f` to terminate the QEMU instance, ensuring each run starts from a clean state.

- **Tracing Orchestration (`run_bench.sh`)**: This script is generated at runtime to configure the kernel's observability features:
  - **Ftrace Setup**: It enables specific scheduler events (`sched_wakeup` and `sched_switch`), configures a monotonic clock (`mono`), and expands the trace buffer to 16MB to prevent data loss.
  - **Execution and Dump**: It iterates through the compiled benchmarks in `/bin`, runs them, and dumps the raw trace data to the serial console for host-side capture.
- **Deterministic Execution**: To minimize non-deterministic jitter, we implement host-level optimizations:
  - **CPU Pinning**: The QEMU process is pinned to a specific host core via `taskset -c 3`.
  - **Hardware Stabilization**: The host CPU governor is set to `performance` and Intel Turbo Boost is disabled to prevent frequency scaling from skewing results.
  - **Resource Constraints**: The VM is limited to a single core (`-smp 1`) and 512MB of RAM, ensuring all task contention is localized and observable.

2.2.2 Differential Analysis and Custom Injection

The Experimental (Custom) pipeline differs only in its initial phase. While the baseline uses the vanilla source, the `build_run_custom.sh` script recursively injects the modified scheduler logic from the project's `src/` directory into the kernel tree before compilation. To maintain strict consistency, the custom run reuses the same `initramfs` generated during the baseline phase, ensuring that the userspace and benchmark binaries are identical across both tests.

2.2.3 Statistical Rigor

To filter out virtualization-induced noise, the pipeline executes as default 20 automated runs for each kernel version. For each run, a custom parser calculates the $p50$, $p95$, and $p99$ wake-up latencies. We then apply a "Median-of-Medians" approach, calculating the median of each percentile across the entire 20-run set. This provides a robust metric that is resistant to transient outliers or virtualization "steal time".

3 Static vruntime boosting

3.1 Modification

In the initial attempts to reduce wake-up latency for I/O bound processes, a direct manipulation of the CFS base logic was implemented. The goal was to manually enforce a leftward shift in the RB Tree for tasks that exhibited interactive behavior (hence the I/O), so to trigger immediate preemption after wakeup.

Inside the `place_entity` function inside the `kernel/sched/fair.c`, instead of relying on the kernel's default "`in_iowait`" flag, we developed a mathematical profile based on context switch counters:

- `p->mm != NULL` (the memory management pointer of the process is active) filters out the kernel threads and background processes (which have a separate kernel memory space).
- `p->nvcsw > 100` checks that the voluntary context switches of the process are enough to categorize it as a probable I/O bound.
- `p->nvcsw > (p->nivcsw * 2)` ensures that the process's voluntary switches are higher than the involuntary ones.

the selected processes, which it was checked with a trace_printk that were the two selected I/O processes only, were given a vruntime subtraction with a specific value (es 0.2 ms, 1 ms, 5 ms.. etc. have been tried out). Although many combinations "filter strictness/vruntime boost" were experimented, none of them resulted in a significant improvement of the p50, p95, and p99 metrics.

3.2 Benchmark

To evaluate the impact of the scheduler modifications, we developed a specialized benchmark suite designed to stress wake-up latency under high CPU contention. The suite, implemented in `benchmarks/test_io.c`, simulates a scenario where latency-sensitive tasks must compete with throughput-oriented workloads on a single core.

3.2.1 Process Archetypes

The benchmark initiates five concurrent processes, each specifically named via `prctl` for identification in the *ftrace* logs:

- **CPU-Bound (3 processes)**: These tasks execute an infinite mathematical loop, ensuring the CPU runqueue is never empty and providing constant pressure on the scheduler.
- **I/O-Bound (`PING` and `PONG`)**: Two synchronized tasks that simulate interactive workloads by frequently blocking on I/O operations.

The `PING` and `PONG` processes communicate via two unidirectional pipes for 5,000 iterations. Each iteration involves a `read()` call that blocks the process, followed by a wake-up triggered by the other process's `write()`. Upon waking, the process performs a short computational burst (`do_work`) and a brief `usleep(5000)`. This specific pattern forces the scheduler to frequently handle wake-up events, allowing us to measure the time elapsed between the `sched_wakeup` event and the actual execution of the task (`sched_switch`).

3.2.2 Observability and parsering of results

The use of `PR_SET_NAME` is critical for the automated parsing pipeline. By labeling processes as `CPU-BOUND` and `IO-BOUND`, the parser can isolate the specific wake-up latencies of the target tasks, providing the data necessary for calculating the p50, p95, and p99 percentiles reported in the final analysis.

To process the significant volume of data generated by *ftrace*, we developed a parser in C (`parse_short.c`). The parser acts as the final stage of the measurement pipeline, transforming raw kernel events into statistical data:

- **Task Identification**: The parser first scans the log for the specific output string `"IO-bound:%d"`, associating PIDs with their functional roles within the benchmark.
- **Latency Calculation**: It monitors the sequence of scheduler events for each target PID. When a `sched_wakeup` occurs, the tool records the timestamp and marks the task as *waiting*. Upon detecting a subsequent `sched_switch` where the target PID becomes the `next_pid`, the parser calculates the wake-up latency as the delta between these two events.
- **Statistical Aggregation**: For each I/O-bound process, the tool collects all calculated samples into a dynamic array. It then utilizes the `qsort` algorithm to organize the data, allowing for the precise calculation of the average, p50 (median), p95, p99, and maximum latencies.

This automated approach ensures that the performance analysis is entirely objective and reproducible, as the same parsing logic is applied consistently to both the baseline and experimental kernel logs.

4 Short block preempt

4.1 Modification

A second attempt to reduce the wakeup-to-scheduling latency focused on modifying the `check_preempt_wakeup` function. The idea was to intervene when the wakee did not manage to preempt the currently executing task. In particular, we gave the wakee a second chance to preempt if:

- it was a recent sleeper, i.e., it had not been off the CPU for longer than `short_sleep_threshold_ns`;
- the current task had executed for at least `sysctl_sched_min_granularity`, in order to prevent starvation.

If the above conditions were satisfied, we checked whether preemption was fair, but using a reduced granularity (`short_sleep_gran_ns`).

To identify recent sleeper tasks, we implemented the helper function `last_offcpu_start`, which returns the timestamp of the wakee's most recent off-CPU interval. In the final implementation, it relies only on `block_start`, so the policy targets wakeups from I/O-like blocking. An optional extension (included but commented out) also considers `sleep_start`, extending the heuristic to voluntary sleepers.

4.1.1 Parameters Tuning and Benchmark

This solution required tuning parameters that were highly dependent on the benchmark. We experimented with a variation of the previously described benchmark: in order to focus on wakeups from I/O sleep, we eliminated the `do_work()` and the forced sleep from the `PING` and `PONG` processes. From the baseline-kernel results, we observed that the median latency was already very low (≈50 µs), while the main issue was the tail of the distribution (p95: ≈1.37 ms, p99: ≈2.25 ms).

Therefore, we decided to target the tail and tried different configurations. With `short_sleep_threshold_ns` set to 300 µs and `short_sleep_gran_ns` set to 250 µs, the results were practically identical to the baseline. Other combinations with small variations of the two parameters (200/500 µs for the former, and 250/150 µs for the latter) did not significantly impact the baseline results. However, a large increase of the threshold (1 ms) led to a significant degradation of the tail. A final attempt involved relaxing the `min_granularity` constraint used to avoid starvation, but this also degraded the tail. We concluded that, in a single-core mixed workload, scheduling stability may matter more than reactivity for tail latency.

5 Adaptive Per-Core Scheduling Policy

In the third phase of the project, we decided to pursue the offered challenge and design an adaptive per core schedulign policy.

The objective is to dynamically bias each CPU either toward responsiveness (for I/O-dominated workloads) or toward throughput (for CPU-dominated workloads), without introducing a new scheduling class or changing CFS's core fairness model.

The design follows a three-stage pipeline: (i) per-task behavioral characterization, (ii) per-core workload aggregation, and (iii) adaptive parameter scaling and enforcement. Each stage is integrated into existing scheduler hooks to minimize overhead and preserve CFS semantics.

5.1 Per-Task Behavioral Characterization

Tasks are classified dynamically based on how they alternate between CPU execution and sleeping. To support this, additional fields are introduced in `task_struct`:

```
u64 ad_sleep_start_ns;
u64 ad_last_sum_exec_runtime;
u64 ad_burst_ewma_ns;
```

```
u64 ad_sleep_ewma_ns;
u8  ad_class;
```

Two smoothed metrics are tracked using an Exponentially Weighted Moving Average (EWMA): the average CPU *burst* (how long the task runs before sleeping) and the average *sleep* time (how long it remains blocked before waking up).

The burst sample is derived when a task interrupts its execution(`DEQUEUE_SLEEP`). It is obtained by taking the difference between the current `se.sum_exec_runtime` and the parameters saved at enqueue time. The sleep sample is computed on wakeup from `rq_clock_task(rq)` and `ad_sleep_start_ns`.

Classification is performed in `ad_classify_task()` using as thresholds values the costants:

```
#define AD_SLEEP_TH_NS  1000000ULL   /* 1 ms */
#define AD_BURST_TH_NS   200000ULL   /* 200 us */
```

A task is marked `AD_IO_LIKE` when it tends to sleep longer and run in short bursts; otherwise it is considered `AD_CPU_BOUND`. The classification is updated in the wakeup path (`ttwu_do_wakeup()`). We restrict the classification to `fair_sched_class` tasks to exclude kernel threads and avoid statistics pollution.

5.2 Per-Core Workload Aggregation

Per-task classifications are aggregated at the runqueue level. For each CPU, a per-core state structure is created:

```
struct ad_cpu_state {
    u64 window_start_ns;
    u32 wakeups;
    u32 io_wakeups;
    u32 io_ratio_ewma;
    u64 min_gran_ns;
    u64 wakeup_gran_ns;
};
```

Whenever a task wakes up, `ad_account_wakeup(rq,p)` increments the counters (`wakeups` and `io_wakeups`). At every fixed observation window (200 ms) (managed in `task_tick_fair()`), the code computes an I/O intensity ratio and applies EWMA smoothing to obtain `io_ratio_ewma`. This value represents how "I/O-heavy" the local core workload is, while filtering short transients.

5.3 Adaptive Parameter Scaling and Enforcement

The adaptive layer maps `io_ratio_ewma` to per-core scheduling knobs. Two CFS parameters are tuned:

- **Minimum granularity** (stored as `min_gran_ns`): the minimum time slice, influencing slice length and context-switch frequency.
- **Wakeup granularity** (stored as `wakeup_gran_ns`): the wakeup-preemption threshold, influencing responsiveness to newly woken tasks.

A bounded scaling factor is derived from the I/O ratio and applied to the global sysctl baselines `sysctl_sched_min_granularity` and `sysctl_sched_wakeup_granularity`. Intuitively, I/O-heavy cores reduce both values to favor quicker preemption and lower wakeup latency, while CPU-heavy cores increase them to reduce scheduling overhead and improve throughput.

To make adaptation effective, global sysctl reads in CFS decision points are replaced with per-core functions:

```
ad_cpu_min_granularity_ns(struct rq *rq)
ad_cpu_wakeup_granularity_ns(struct rq *rq)
```

These helpers return the locally tuned values when available and fall back to the sysctl defaults otherwise. They are used in wakeup preemption logic (e.g., `wakeup_gran()` / `check_preempt_wakeup()`) and slice clamping, where a minimum granularity is enforced.

5.4 Benchmark Adaptation for Phase-Controlled Workloads

To evaluate the effectiveness of the adaptive per-core policy under controlled and reproducible workload variations, the original benchmark introduced in the previous section was extended.

The benchmark maintains a configurable and constant number of CPU-bound worker processes, while the behavior of I/O-bound processes has been refined to introduce explicit workload phases. In particular, each I/O-bound process alternates between two distinct behavioral modes:

- **I/O-active phase**: the process performs pipe-based communication (ping-pong interaction), alternating between short CPU bursts and blocking I/O operations.
- **I/O-sleep phase**: the process enters a significantly longer sleep interval without performing active communication.

This design allows the definition of a configurable number of workload phases. During I/O-active phases, frequent wakeups increase the measured I/O ratio on the target core, whereas during I/O-sleep phases the wakeup rate drops significantly, reducing the perceived I/O intensity.

To introduce an additional degree of control, the duration of the I/O-sleep phase was made dependent on both:

- the current phase index, and

- the CPU on which the task is running.

This two-dimensional parameterization enables the generation of core-specific workload patterns, effectively stressing the adaptive mechanism by creating heterogeneous per-core I/O intensities within the same experiment.

As a result, the benchmark provides deterministic phase transitions that directly impact the aggregated per-core I/O ratio. This makes it possible to observe whether the adaptive scaling of minimum granularity and wakeup granularity reacts coherently to workload changes.

5.5 Experimental Infrastructure and Automated Analysis

To support these experiments, the build and execution pipeline was adapted thanks to a runtime script inside the initramfs that executes the benchmarks pinned to specific virtual CPUs.

Experiments are conducted on a two-core QEMU virtual machine (`-smp 2`) to observe per-core behavior.

To analyze the results, a dedicated script (`ad_summary.sh`) processes the raw execution log and extracts: (i) classification accuracy from `AD_CLASS` debug prints, (ii) per-core time series of `io_ratio_ewma`, `min_gran_ns`, and `wakeup_gran_ns`, and (iii) ground-truth workload phase reconstruction from benchmark markers. Each scheduler sample is labeled according to the active workload phase.

A companion Python visualization tool then generates per-core plots where background shading represents workload phases and lines represent the evolution of adaptive scheduling parameters.

5.6 Results of adaptive policy

Figure 1 reports the per-core evolution of `io_ratio_ewma`, `min_gran_ns`, and `wakeup_gran_ns` under phase-controlled workload variations.

First, the task-level classification mechanism proved to be highly accurate. Across all experiments, the classification accuracy remained consistently above 98%, confirming that the EWMA-based burst and sleep tracking reliably distinguishes I/O-like and CPU-bound behaviors. This high accuracy ensures that per-core aggregation is not distorted by systematic misclassification.

From a qualitative perspective, the adaptive response closely follows workload transitions. During I/O-active phases (frequent wakeups and short bursts), the per-core `io_ratio_ewma` increases, and both `min_gran_ns` and `wakeup_gran_ns` are reduced. This behavior favors responsiveness by allowing quicker preemption and shorter minimum slices. Conversely, during extended I/O-sleep phases, the I/O ratio decreases and the scheduler progressively increases both granularity parameters, reducing context-switch overhead and favoring throughput.

Importantly, the two cores exhibit differentiated behavior when subjected to heterogeneous phase patterns. Since the benchmark introduces CPU-specific phase durations, the adaptive

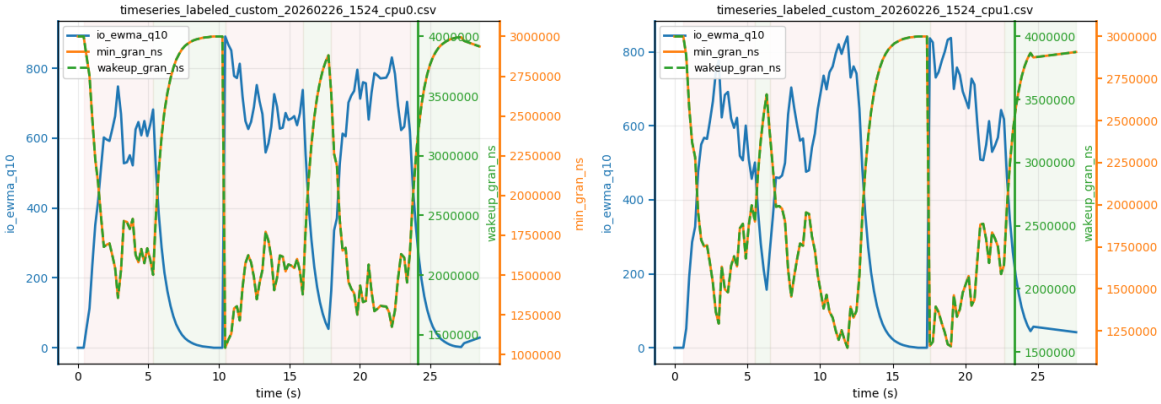

Figure 1: Per-core evolution of `io_ratio_ewma`, `min_gran_ns`, and `wakeup_gran_ns` across workload phases.

parameters evolve independently on each core. This demonstrates that adaptation is truly per-core rather than globally synchronized.

6 Conclusions

This project provided hands-on experience with the Linux Completely Fair Scheduler (CFS) and demonstrated how targeted modifications can influence system responsiveness and throughput.

Through three complementary experiments—viruntime-based I/O boosting, preemption-based I/O boosting, and adaptive per-core parameter scaling—we explored different design dimensions of scheduler behavior. Each approach highlighted how specific CFS mechanisms (e.g., wakeup preemption, virtual runtime accounting, granularity control) can be tuned to favor either interactive or compute-intensive workloads while preserving fairness semantics.

Beyond the scheduling logic itself, the project required building a complete experimental pipeline: kernel recompilation under QEMU, custom initramfs generation, benchmark design with controlled workload phases, automated log parsing via Bash scripts, and structured result visualization in Python.

Overall, the work deepened our understanding of CFS internals and demonstrated that adaptive scheduling strategies can be implemented incrementally, leveraging existing hooks and parameters without introducing new scheduling classes.